

Deep Research: Avatrada 57 Topic 046

Engineering Report: Auto-healing MIG for a High-Availability Trading Server

1. Technical Deconstruction

This analysis deconstructs the architecture of a self-healing Managed Instance Group (MIG) on Google Cloud Platform (GCP), designed for a stateful application like a trading server. The core principle is to combine stateless compute instances with a stateful persistent disk, orchestrated by an application-aware Health Check.

1.1 System Components & Interaction

The system is composed of four primary GCP components that interact in a specific lifecycle:

1. **Instance Template:** This is the blueprint for the trading server VM. It defines the machine type, boot disk image (e.g., Debian 11), and crucially, the configuration for attaching an *existing* data disk. It also contains a startup script responsible for installing dependencies, mounting the data disk, and launching the trading application.
2. **Persistent Disk (PD):** This is a network-attached block storage device that exists independently of any VM. It is configured as a *data disk* (not a boot disk) and will store the application's stateful data, such as SQLite databases, log files, or configuration files. Its key property for this architecture is that its lifecycle is decoupled from the VM; it is not deleted when the VM is terminated.
3. **Health Check:** This is a proactive monitoring service. For this use case, we employ an HTTP-based Health Check. It periodically sends an HTTP GET

request to a specific endpoint on the trading server (e.g., `http://<instance-ip>:8000/health`).

- **Success Condition:** The server responds with an `HTTP 200 OK` status code within a defined timeout period.
- **Failure Condition:** The server responds with a non-200 status, or more relevant to a deadlock, it fails to respond entirely, causing the check to time out.

4. **Managed Instance Group (MIG):** This is the orchestrator. It uses the Instance Template to create and manage one or more identical VMs. Its auto-healing policy is the central logic:

- The MIG associates the Health Check with its instances.
- It continuously monitors the health status reported by the Health Check for each instance.
- If an instance is marked `UNHEALTHY` for a configured number of consecutive checks (the `unhealthyThreshold`), the MIG's auto-healing policy is triggered.

1.2 The Auto-healing and State Preservation Lifecycle

The process unfolds as follows, triggered by an application deadlock:

1. **Normal Operation:** The trading server VM is running. The FastAPI application is responsive. The Health Check service pings the `/health` endpoint every `N` seconds and receives a `200 OK` response. The instance status is `HEALTHY`. The Persistent Disk is mounted and actively used by the application.
2. **Failure Event (Deadlock):** The trading application enters a deadlock state. The FastAPI server, running in the same process, becomes completely unresponsive and can no longer handle incoming HTTP requests.
3. **Health Check Failure:** The GCP Health Check sends its next GET request to `/health`. The request times out. This is recorded as the first failure. This repeats for two more consecutive checks.

4. **Threshold Breach:** After the third consecutive failure (`unhealthyThreshold = 3`), the Health Check service officially marks the instance as `UNHEALTHY`.
5. **MIG Action (Recreation):** The MIG's auto-healing policy detects the `UNHEALTHY` status. It initiates the `REPAIRING` process:
- a. **Termination & Detach:** The MIG sends a termination signal to the unhealthy VM. As part of this process, it automatically *detaches* the stateful Persistent Disk. The boot disk is destroyed with the VM.
 - b. **Creation & Attach:** The MIG immediately creates a new VM using the original Instance Template. During creation, it *attaches* the same, pre-existing Persistent Disk to the new instance.
 - c. **Initialization:** The new VM boots up. The startup script defined in the Instance Template executes. This script mounts the Persistent Disk to the filesystem (e.g., at `/mnt/data`).
 - d. **Application Start:** The startup script then launches the trading application. The application starts, finds its database and log files on the mounted Persistent Disk, and resumes operation from its last known state.
6. **Return to Normal:** The new instance's application is now running correctly. The Health Check begins probing it and receives `200 OK` responses. The instance is marked `HEALTHY`, and the cycle is complete.
-

2. Implementation Strategy

This section provides the step-by-step commands and code to build the described system.

Step 1: The FastAPI Application (`/health` endpoint)

Create a simple Python application. The `/health` endpoint is critical.

```
# main.py
from fastapi import FastAPI
import uvicorn
import logging
import time
```

```

# Assume logs and db are written to /mnt/data
LOG_FILE = "/mnt/data/trading_bot.log"
DATABASE_FILE = "/mnt/data/trades.db"

# Setup basic logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler(LOG_FILE),
        logging.StreamHandler()
    ]
)

app = FastAPI()

@app.get("/health")
def health_check():
    """
    Simple health check endpoint.
    In a real system, this could check DB connectivity.
    """
    return {"status": "ok"}

@app.get("/")
def root():
    logging.info(f"Root endpoint accessed. Pretending to check DB at {DATABASE_FILE}.")
    # Simulate some work
    time.sleep(1)
    return {"message": "Trading bot is active."}

if __name__ == "__main__":
    # Ensure the data directory exists (though the mount should handle this)
    import os
    os.makedirs(os.path.dirname(LOG_FILE), exist_ok=True)

    logging.info("Starting Trading Bot Server...")
    # Run on 0.0.0.0 to be accessible from outside the container/VM
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

Step 2: Prepare the Persistent Disk and Startup Script

First, create the disk that will hold the state.

```
# Create a 10GB persistent disk for our state
gcloud compute disks create trading-server-data-disk \
  --size=10GB \
  --type=pd-standard \
  --zone=us-central1-a
```

Next, create the startup script. This script will be embedded in the instance template. It prepares the disk on first boot and runs the application.

```
# startup-script.sh
#!/bin/bash
set -e # Exit on any error

# --- Disk Mounting ---
# The device name is what we specify in the instance template
# It will appear as /dev/sdb on the system
DATA_DISK_DEVICE="/dev/sdb"
MOUNT_POINT="/mnt/data"

# Check if the disk is already formatted
# Use file -s to get filesystem type. If it's just "data", it's unformatted.
if [ "$(file -s ${DATA_DISK_DEVICE} | awk '{print $2}')" == "data" ]; then
  echo "Formatting data disk ${DATA_DISK_DEVICE}..."
  mkfs.ext4 -m 0 -E lazy_itable_init=0,lazy_journal_init=0,discard $
{DATA_DISK_DEVICE}
fi

# Create mount point and mount the disk
mkdir -p ${MOUNT_POINT}
mount -o discard,defaults ${DATA_DISK_DEVICE} ${MOUNT_POINT}
chmod a+w ${MOUNT_POINT} # Make it writable by any user

# Add to /etc/fstab for auto-mounting on subsequent reboots (not strictly
necessary for MIG recreation, but good practice)
```

```

echo "${DATA_DISK_DEVICE} ${MOUNT_POINT} ext4 defaults,discard 1 1" >> /etc/
fstab

# --- Application Setup ---
echo "Installing dependencies..."
apt-get update
apt-get install -y python3-pip python3-venv

# Create a directory for the app
APP_DIR="/opt/trading-app"
mkdir -p ${APP_DIR}
cd ${APP_DIR}

# Copy your application code here (for a real setup, you'd pull from a repo)
# For this example, we'll just write it directly
cat <<EOF > main.py
from fastapi import FastAPI
import uvicorn
import logging
import time

LOG_FILE = "/mnt/data/trading_bot.log"
DATABASE_FILE = "/mnt/data/trades.db"

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler(LOG_FILE),
        logging.StreamHandler()
    ]
)

app = FastAPI()

@app.get("/health")
def health_check():
    return {"status": "ok"}

@app.get("/")

```

```

def root():
    logging.info(f"Root endpoint accessed. Pretending to check DB at
{DATABASE_FILE}.")
    time.sleep(1)
    return {"message": "Trading bot is active."}

if __name__ == "__main__":
    import os
    os.makedirs(os.path.dirname(LOG_FILE), exist_ok=True)
    logging.info("Starting Trading Bot Server...")
    uvicorn.run(app, host="0.0.0.0", port=8000)
EOF

# Setup Python environment and install packages
python3 -m venv venv
source venv/bin/activate
pip install fastapi uvicorn

# --- Run Application ---
echo "Starting application..."
# Run the app in the background using the virtual environment's python
/opt/trading-app/venv/bin/python /opt/trading-app/main.py > /var/log/app.log
2>&1 &

```

Step 3: Create the Instance Template

This template combines the VM configuration, the persistent disk attachment, and the startup script.

```

gcloud compute instance-templates create trading-server-template \
  --machine-type=e2-micro \
  --image-family=debian-11 \
  --image-project=debian-cloud \
  --scopes=https://www.googleapis.com/auth/cloud-platform \
  --zone=us-central1-a \
  --metadata-from-file=startup-script=startup-script.sh \
  --disk=name=trading-server-data-disk,device-name=data-
disk,mode=rw,boot=no,auto-delete=no

```

```
# Key flags:
# --disk=... : Attaches our stateful disk.
# device-name=data-disk : An arbitrary name we use to reference it. It will map
to /dev/sdb.
# boot=no : This is a data disk, not a boot disk.
# auto-delete=no : CRITICAL. This prevents the disk from being deleted when the
VM is.
```

Step 4: Create the Firewall Rule and Health Check

We need to allow traffic to port 8000 for the health checker.

```
# Allow traffic on port 8000
gcloud compute firewall-rules create allow-http-8000 \
  --allow=tcp:8000 \
  --source-ranges=130.211.0.0/22,35.191.0.0/16 \
  --network=default

# Create the health check
gcloud compute health-checks create http trading-server-health-check \
  --port=8000 \
  --request-path=/health \
  --check-interval=10s \
  --timeout=5s \
  --unhealthy-threshold=3 \
  --healthy-threshold=2
```

- `source-ranges` : These are the specific IP ranges used by GCP's health checkers.
- `unhealthy-threshold=3` : The instance will be marked unhealthy after 3 consecutive failures ($3 * 10s = 30s$ minimum detection time).

Step 5: Create the Managed Instance Group

Finally, create the MIG using the template and the health check.


```
gcloud compute instance-groups managed create trading-server-mig \
  --base-instance-name=trading-server \
  --size=1 \
  --template=trading-server-template \
  --zone=us-central1-a \
  --health-check=trading-server-health-check \
  --initial-delay=120

# Key flags:
# --size=1 : We only want one instance for this stateful setup.
# --health-check : Links our health check to the MIG.
# --initial-delay=120 : CRITICAL. Wait 120 seconds after a VM starts before
beginning health checks.
# This gives the startup script time to install dependencies and launch the app.
```

3. Critical Analysis

This architecture provides a robust solution for automated recovery from application-level failures. However, it is not without its trade-offs and potential failure modes.

3.1 Potential Failure Modes & Edge Cases

1. **State Corruption on Crash:** If the trading application crashes abruptly (e.g., due to the MIG terminating it), it might leave the database or log files in a corrupted state. The new instance will boot, mount the disk, and the application may fail to start because it cannot read the corrupted state.
 - **Mitigation:** Employ robust application design. Use database transactions, write-ahead logging (WAL), and journaling filesystems (like ext4) to ensure state consistency. The application's startup logic should include a routine to check for and recover from a "dirty" shutdown.

2. **Recovery Time Objective (RTO):** The total downtime during a recreation event is non-trivial. It includes:

- Health check failure detection time (`unhealthyThreshold` * `checkInterval` = 3 * 10s = 30s).
- VM termination and creation time (~60-90s).
- Startup script execution time (can be several minutes if installing many packages).
- `initial-delay` before the new instance is considered active (120s).
- **Total RTO:** ~4-5 minutes. For a high-frequency trading server, this is a significant outage.

3. **Health Check Flapping:** A transient network issue or a temporary application overload could cause a few health checks to fail, triggering an unnecessary and lengthy recreation cycle.

- **Mitigation:** Tune the health check parameters carefully. A slightly longer `checkInterval` or a higher `unhealthyThreshold` can prevent flapping at the cost of a longer RTO. Make the `/health` endpoint lightweight and not dependent on external services that could fail independently of the bot's core logic.

4. **"Split-Brain" Risk (Low):** GCP's control plane is designed to prevent a `ReadWriteOnce` disk from being attached to two VMs simultaneously. However, in a catastrophic control plane failure, there is a theoretical risk. The primary real-world risk is the old VM not fully releasing its lock on the disk before the new one tries to mount it, causing I/O errors on the new instance. GCP's orderly detach/attach process makes this highly unlikely.

5. **Crash Loops:** If the failure is caused by a bug in the application code or a misconfiguration in the startup script that is present in the Instance Template, the MIG will enter a crash loop. It will recreate the VM, the new VM will fail for the same reason, and the cycle will repeat.

- **Mitigation:** Thoroughly test new instance templates before rolling them out. Use GCP's monitoring and alerting to detect high rates of instance recreation.

3.2 Optimizations and Alternative Architectures

1. **Pre-baking Images:** The startup script installs dependencies on every boot, increasing the RTO. A significant optimization is to create a custom machine image with all dependencies and the application code pre-installed. The startup script would then only need to mount the disk and run the application, reducing boot time from minutes to seconds.
2. **State Decoupling (Superior Architecture):** The most robust and scalable solution is to completely decouple state from the instance.
 - **Database:** Instead of a local SQLite file on a Persistent Disk, use a managed database like **Cloud SQL** or **Spanner**. The application instance connects to this external database.
 - **Logs:** Instead of writing to a local file, stream logs directly to **Cloud Logging**.
 - **Result:** With this model, the VM becomes truly stateless. If it fails, the MIG can recreate it in seconds, and the new instance simply reconnects to the external database and logging services. This eliminates the complexities of disk attachment/detachment and dramatically reduces the RTO. This is the professionally recommended pattern for high-availability services.
3. **Containerization (GKE):** For even faster recovery, containerize the trading application and run it on Google Kubernetes Engine (GKE). A Kubernetes `StatefulSet` can manage a pod with an attached Persistent Disk. Pod rescheduling is significantly faster than VM recreation, potentially reducing RTO to under a minute. However, this introduces the complexity of Kubernetes. The stateless pattern (using Cloud SQL) is still preferable even within GKE.